Software Testing & Quality Assurance (23UAIOEL3602A)

Unit IV: Acceptance Testing and Software Quality

Unit IV: Acceptance Testing and Software Quality
Quality Concepts and Review Techniques (Inspections & Walkthroughs). They delve into Software Quality Assurance (SQA), including Verification and Validation, SQA Plans, and quality frameworks. Key content focuses on Quality Assurance Models like the ISO 9000 series, CMM, CMMI, Test Maturity Models, and SPICE. Finally, advanced SQA trends are discussed, such as PSP, TSP, Clean-room engineering, defect prevention, Internal Auditing

Software Quality is the degree to which software meets specified requirements and user expectations.

Acceptance testing is the final phase of software testing, ensuring a product meets business requirements and is ready for production.

Conducted by stakeholders, end-users, or clients, it validates that the system functions as intended in real-world scenarios.

- Key types include User Acceptance Testing (UAT),
- Operational Acceptance Testing (OAT),
- Contract Acceptance Testing (CAT).

Key Aspects of Acceptance Testing:

To confirm the software fulfills user needs and passes the "go/no-go" decision point before release.

Key Types:

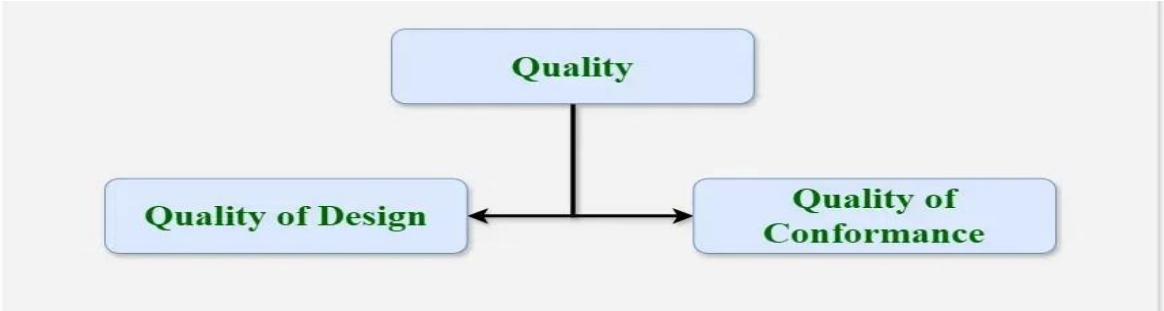
- User Acceptance Testing (UAT): Real users test the software to ensure it handles daily tasks in real-world scenarios.
- Operational Acceptance Testing (OAT): Validates the system's operational readiness, including compatibility, stability, and reliability.
- Alpha Testing: Performed by internal teams at the developer's site.
- Beta Testing: Conducted by end-users in their own environment before official release.
- Contract Acceptance Testing (CAT): Ensures the product meets the specific criteria defined in a contract.
- Regulations Acceptance Testing (RAT): Ensures compliance with legal regulations.

1.Software Quality Assurance - Software Engineering(SQA)

Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly. It is a process that works parallel to Software Development.

It focuses on improving the process of development of software so that problems can be prevented before they become major issues.

Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.



Software quality refers to the **degree to which a software product meets user requirements and expectations**. A high-quality software system is correct, reliable, efficient, easy to use, and maintainable.

Quality in software is achieved through two main approaches: **Quality Assurance (QA)** and **Quality Control (QC)**. Quality Assurance is a **process-oriented activity** that focuses on preventing defects by following proper standards and procedures during development. Quality Control is a **product-oriented activity** that focuses on detecting defects through testing and review.

2.Review Techniques (Inspections & Walkthroughs)

Review techniques are static testing methods used to evaluate software without executing it. They help in detecting defects early and improving software quality.

Inspection: is a formal and structured review technique used to identify defects in documents, design, or code. It follows a defined process that includes planning, preparation, inspection meeting, rework, and follow-up.

Inspections involve specific roles such as moderator, author, reviewers, and recorder. The main objective is to detect defects early and ensure adherence to standards.

Walkthrough: is an informal review technique in which the author presents the software to team members. It is mainly used for understanding the system and gathering feedback.

Participants discuss the content, ask questions, and suggest improvements. Walkthroughs are easy to conduct but less effective than inspections in finding complex defects.

Software Quality Assurance (SQA) :

Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly. It is a process that works parallel to Software Development.

It focuses on improving the process of development of software so that problems can be prevented before they become major issues. Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.

Elements of Software Quality Assurance (SQA)

- **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
- **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
- **Testing:** Software testing is a quality control function that has one primary goal to find errors.
- **Error/defect collection and analysis** : SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change management:** SQA ensures that adequate change management practices have been instituted.
- **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.

- **Security management:** SQA ensures that appropriate process and technology are used to achieve software security.
- **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
- **Risk management** : The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established

Focus of Software Quality Assurance (SQA)

- **Software's Portability:** Software's portability refers to its ability to be easily transferred or adapted to different environments or platforms without needing significant modifications.
- **Software's Usability:** Usability of software refers to how easy and intuitive it is for users to interact with and navigate through the application.
- **Software's Reusability:** Reusability in software development involves designing components or modules that can be reused in multiple parts of the software or in different projects.
- **Software's Correctness:** Correctness of software refers to its ability to produce the desired results under specific conditions or inputs.
- **Software's Maintainability:** Maintainability of software refers to how easily it can be modified, updated, or extended over time.

3.Verification and Validation:

Verification and Validation is the process of investigating whether a software system satisfies specifications and standards and fulfills the required purpose. Verification and Validation both play an important role in developing good software development. Verification helps in examining whether the product is built right according to requirements, while validation helps in examining whether the right product is built to meet user needs.

What is Verification?

Verification is the process of ensuring that the software work products (such as requirements, design, and code) conform to specified requirements and standards. It focuses on checking correctness and consistency without necessarily executing the software.

What is Validation?

Validation is the process of evaluating the software during or after development to ensure it fulfills its intended use and meets user and stakeholder needs. It typically involves executing the software and observing its behaviour in real or simulated environments.

Advantages of Differentiating Verification and Validation

Differentiating between verification and validation in software testing offers several advantages:

1. **Clear Communication:** It ensures that team members understand which aspects of the software development process are focused on checking requirements (verification) and which are focused on ensuring functionality (validation).
2. **Efficiency:** By clearly defining verification as checking documents and designs without executing code, and validation as testing the actual software for functionality and usability, teams avoid redundant efforts and streamline their testing processes.
3. **Minimized Errors:** It reduces the chances of overlooking critical requirements or functionalities during testing, leading to a more thorough evaluation of the software's capabilities.
4. **Cost Savings:** Optimizing resource allocation and focusing efforts on the right testing activities based on whether they fall under verification or validation helps in managing costs effectively.

What is a Software Quality Assurance Plan:

A Quality Assurance Plan (QAP) is a document or set of documents that outlines the systematic processes, procedures, and standards for ensuring the quality of a product or service.

It is a key component of quality management and is used in various industries to establish and maintain a consistent level of quality in deliverables.

An SQA plan will include several components, such as purpose, references, configuration and management, tools, code controls, testing methodology, problem reporting and remedial measures, and more, for easy documentation and referencing.

Importance of Software Quality Assurance Plan

- **Quality Standards and Guidelines:** The SQA Plan lays out the requirements and guidelines to make sure the programme satisfies predetermined standards for quality.
- **Risk management:** It is the process of recognizing, evaluating and controlling risks in order to reduce the possibility of errors and other problems with quality.
- **Standardization and Consistency:** The strategy guarantees consistent methods, processes, and procedures, fostering a unified and well-structured approach to quality assurance.
- **Customer Satisfaction:** The SQA Plan helps to ensure that the finished product satisfies customer needs, which in turn increases overall customer satisfaction.
- **Resource optimization:** It is the process of defining roles, responsibilities, and procedures in order to maximize resource utilization and minimize needless rework.

.

Quality frameworks:

Quality frameworks are structured systems of policies and processes designed to ensure that products and services consistently meet defined standards of excellence.

They provide a roadmap for organizations to enhance customer satisfaction, achieve operational efficiency, and maintain regulatory compliance.

Key Quality Assurance Models and Frameworks:

1. ISO 9000 Series (ISO 9001): An international standard for Quality Management Systems (QMS) that focuses on documentation, process auditing, and continual improvement.

The ISO 9000 Series is a set of international standards for Quality Management Systems (QMS) developed by the International Organization for Standardization. These standards provide guidelines and principles to ensure that organizations consistently produce high-quality products and services that meet customer and regulatory requirements.

The ISO 9000 series is not limited to software; it is applicable to all types of industries, including manufacturing and service sectors.

Objectives of ISO 9000 Series

The main objectives are:

- To ensure consistent quality of products and services
- To improve customer satisfaction
- To establish standardized processes
- To promote continuous improvement
- To ensure compliance with international standards

2. Capability Maturity Model (CMM)

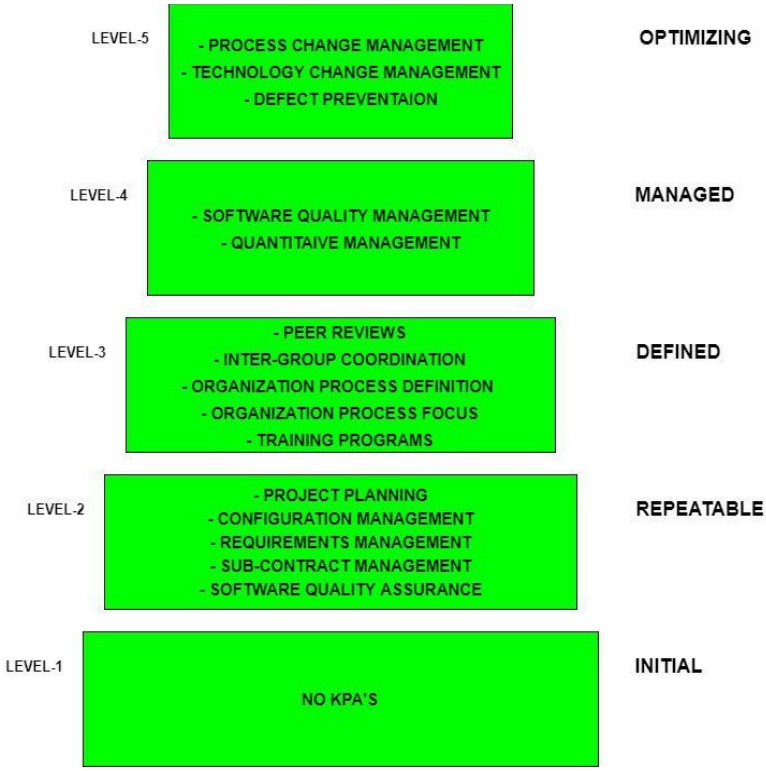
Capability Maturity Model (CMM) was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987.

NOTE: It is not a software process model. It is a framework that is used to analyze the approach and techniques followed by any organization to develop software products. It also provides guidelines to enhance further the maturity of the process used to develop those software products.

Importance of Capability Maturity Model

- **Optimization of Resources:** CMM helps businesses make the best use of all of their resources, including money, labor, and time. Organizations can improve the effectiveness of resource allocation by recognizing and getting rid of unproductive practices.
- **Comparing and Evaluating:** A formal framework for benchmarking and self-evaluation is offered by CMM.
- **Management of Quality**: CMM emphasizes quality management heavily. The framework helps businesses apply best practices for quality assurance and control, which raises the quality of their goods and services.
- **Enhancement of Process**: CMM gives businesses a methodical approach to evaluate and enhance their operations.
- **Increased Output:** CMM seeks to boost productivity by simplifying and optimizing processes. Organizations can increase output and efficiency without compromising quality as they go through the CMM levels.

Levels of Capability Maturity Model (CMM):



Level-1: Initial

- No KPIs defined. ((Key Performance Indicators))
- Processes followed are Adhoc and immature and are not well defined.
- Unstable environment for software development.
- No basis for predicting product quality, time for completion, etc.
- Limited project management capabilities, such as no systematic tracking of schedules, budgets, or progress.

Level-2: Repeatable

Focuses on establishing basic project management policies.

Experience with earlier projects is used for managing new similar-natured projects.

- **Project Planning-** It includes defining resources required, goals, constraints, etc. for the project. It presents a detailed plan to be followed systematically for the successful completion of good-quality software.

Level-3: Defined

At this level, documentation of the standard guidelines and procedures takes place.

- It is a well-defined integrated set of project-specific software engineering and management processes.
- **Peer Reviews:** In this method, defects are removed by using several review methods like walkthroughs, inspections, buddy checks, etc.
- **Intergroup Coordination:** It consists of planned interactions between different development teams to ensure efficient and proper fulfilment of customer needs.
- .

Level-4: Managed

At this stage, quantitative quality goals are set for the organization for software products as well as software processes.

The measurements made help the organization to predict the product and process quality within some limits defined quantitatively.

- **Software Quality Management:** It includes the establishment of plans and strategies to develop quantitative analysis and understanding of the product's quality.
- **Quantitative Management:** It focuses on controlling the project performance quantitatively.

Level-5: Optimizing

This is the highest level of process maturity in CMM and focuses on continuous process improvement in the organization using quantitative feedback.

The use of new tools, techniques, and evaluation of software processes is done to prevent the recurrence of known defects.

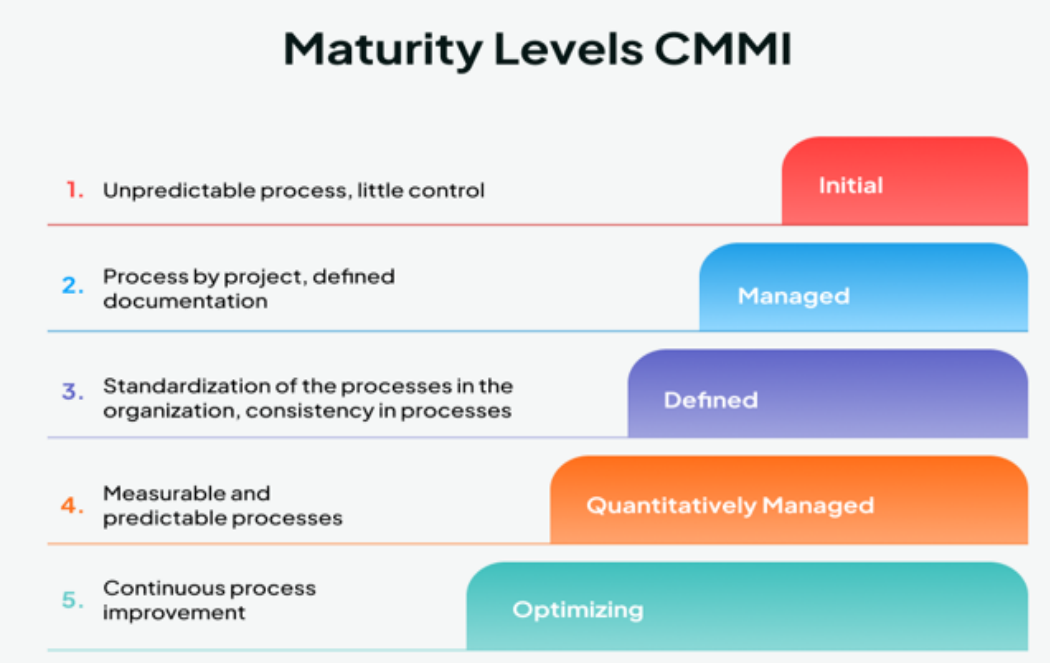
- **Process Change Management:** Its focus is on the continuous improvement of the organization's software processes to improve productivity, quality, and cycle time for the software product.
- **Technology Change Management:** It consists of the identification and use of new technologies to improve product quality and decrease product development time.
-

3. Capability Maturity Model Integration (CMMI)

Capability Maturity Model Integration (CMMI) is a successor of CMM and is a more evolved model that incorporates best components of individual disciplines of CMM like Software CMM, Systems Engineering CMM, People CMM, etc. Since CMM is a reference model of matured practices in a specific discipline, so it becomes difficult to integrate these disciplines as per the requirements. This is why CMMI is used as it allows the integration of multiple disciplines as and when needed.

Objectives of CMMI

- Fulfilling customer needs and expectations.
- Value creation for investors/stockholders.
- Market growth is increased.
-



Importance of Capability Maturity Model

Capability Maturity Model Integration (CMMI) is a process improvement framework that helps organizations improve their software development and business processes. It provides a structured approach to enhance quality, efficiency, and performance by defining best practices.

CMMI was developed by the Software Engineering Institute to guide organizations in improving their process maturity.

Maturity Levels in CMMI

CMMI defines **five maturity levels** that indicate the capability of an organization's processes:

Level 1 – Initial

- Processes are **unpredictable and unorganized**
- Success depends on individual efforts

Level 2 – Managed

- Basic **project management practices** are established
- Processes are planned and controlled

Level 3 – Defined

- Processes are **standardized and documented**
- Organization follows uniform procedures

Level 4 – Quantitatively Managed

- Processes are **measured and controlled using data**
- Performance is predictable

Level 5 – Optimizing

- Focus on **continuous improvement**
- Defects are prevented through innovation and feedback

SPICE:

SPICE (ISO/IEC 15504) is an **international standard for assessing and improving software processes**. It provides a framework to evaluate the **capability and maturity of processes** within an organization.

- SPICE helps organizations to:
- Identify strengths and weaknesses in processes

- Improve process capability
- Ensure consistent software quality

Key Features of SPICE

- Focuses on **process assessment and improvement**
- Defines **capability levels** for processes
- Provides a structured method for **process evaluation**
- Aligns with international quality standards

Capability Levels in SPICE:

SPICE defines six capability levels:

- **Level 0 – Incomplete** → Process not implemented
- **Level 1 – Performed** → Process achieves its purpose
- **Level 2 – Managed** → Process is planned and controlled
- **Level 3 – Established** → Process is standardized
- **Level 4 – Predictable** → Process is measured and controlled

Advanced SQA Trends:

Modern Software Quality Assurance includes advanced techniques to improve quality and reduce defects.

1. Personal Software Process(PSP)

- PSP is a method that helps individual developers improve their work quality by following a structured process.

Focus on personal planning and tracking

- Improves coding and testing practices
- Encourages self-discipline and quality awareness

2. Team Software Process(TSP)

- TSP extends PSP to the team level, helping teams work efficiently and deliver high-quality software.

Features:

- Emphasizes team planning and coordination
- Improves productivity and quality
- Encourages team responsibility

3. Cleanroom Software Engineering(CSE)

- Cleanroom engineering is a technique focused on **defect prevention rather than defect removal**.

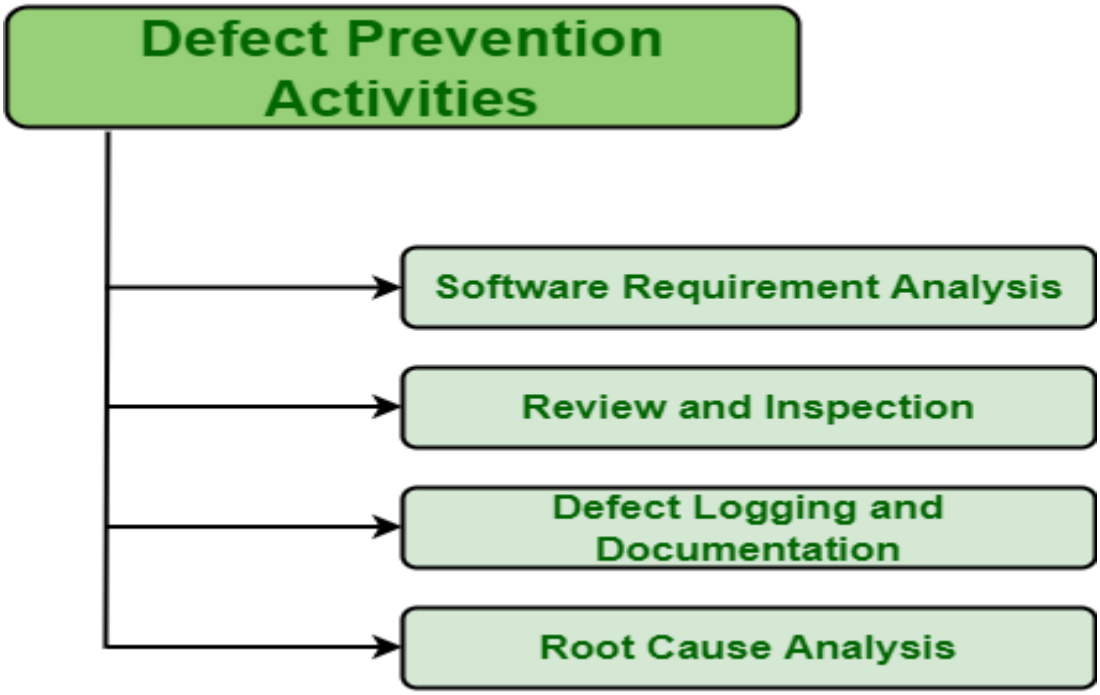
Features:

- Uses **formal methods for design and verification**
- Avoids debugging; aims for **zero-defect software**
- Relies on **statistical testing**

Defect Prevention:

Defect prevention is not merely about identifying and fixing issues after they occur; it's about proactively implementing measures to stop defects from happening in the first place.

This approach saves time, resources, and maintains customer satisfaction by delivering products that meet or exceed expectations right from the start.



1. **Software Requirement Analysis :** The main cause of defects in software products is due to error in software requirements and designs. Software requirements and design both are important, and should be analyzed in an efficient way with more focus. Software requirement is basically considered an integral part of Software Development Life Cycle (SDLC). These are the

requirements that basically describes features and functionalities of target product and also conveys expectations or requirement of users from software product.

2. **Review and Inspection :**

   Review and inspection, both are essential and integral part of software development. They are considered as powerful tools that can be used to identify and remove defects if present before their occurrence and impact on production.

   Review and inspection come in different levels or stages of defect prevention to meet different needs. They are used in all software development and maintenance methods. There are two types of review i.e., self-review and peer-review.

3. **Defect Logging and Documentation :**

   After successful analysis and review, there should be records maintained about defects to simply complete description of defect.

   This record can be further used to have better understanding of defects. After getting knowledge and understanding of defect, then only one can take some effective and required measures and actions to resolve particular defects so that defect cannot be carried further to next phase.

4. **Root Cause Analysis :** Root because analysis is basically analysis of main cause of defect. It simply analysis what triggered defect to occur. After analyzing main cause of defect, one can find best way to simply avoid occurrence of such types of defects next time.

Defect prevention focuses on identifying root causes of defects and eliminating them.

Techniques:

- Root cause analysis
- Process improvement
- Training and standards

Defect prevention using Root Cause Analysis (RCA) and process improvement is a proactive approach to quality management, aiming to eliminate the fundamental reasons for defects rather than merely fixing symptoms.

By analysing past failures, organizations can implement targeted improvements, reduce rework costs, increase efficiency, and foster a culture of continuous improvement.

Core Components of Defect Prevention

- **Proactive Strategy**: Defect prevention involves analysing past defects to take action against future occurrences, moving away from reactive "firefighting".
- **Root Cause Analysis (RCA):** A structured process—including tools like the "5 Whys" and Fishbone Diagrams—to identify the fundamental cause of a non-conformance.
- Process Improvement (PI): Changes to procedures, tools, training, or standards to strengthen the overall system and prevent defect recurrence.
- Blameless Culture: Focusing on fixing processes rather than blaming individuals is crucial for effective, honest reporting and analysis.

Key RCA Tools:

1. 5 Whys: Repeatedly asking "why" to dig down to the root cause.
2. Fishbone (Ishikawa) Diagram: A visual tool for brainstorming potential causes.
3. Pareto Analysis: A 80/20 rule application that helps prioritize the most significant causes of defects.
4. Failure Mode and Effects Analysis (FMEA): A systematic method to identify potential failure modes and their risks.
5. Orthogonal Defect Classification (ODC): A method for classifying defects to pinpoint process gap

Internal Auditing

- Internal auditing is the process of **reviewing and evaluating organizational processes** to ensure compliance with standards.
- **Features:**
- Conducted within the organization
- Checks adherence to **quality procedures**
- Identifies areas for improvement

Internal auditing in software testing is an in-house evaluation of testing processes, tools, and documentation to ensure they align with established standards, compliance requirements (e.g., ISO, GDPR), and quality goals.

It identifies inefficiencies, such as poor test coverage or outdated test cases, to reduce risks and improve software reliability before release.

Key Aspects of Internal Audits in Software Testing:

Objectives: Evaluate compliance, identify security risks, assess software licensing, and optimize testing processes.

- **Process:**
    1. **Planning:** Define the audit scope, objectives, and schedule.
    2. **Fieldwork:** Conduct interviews, review documentation (policies, logs), and observe tests.
    3. **Analysis:** Evaluate actual practices against standards (benchmarking).
    4. **Reporting:** Document findings, including gaps, root causes, and recommendations (5 Cs: Criteria, Condition, Cause, Consequence, Corrective action).
- **Common Types:**
    - **Process Audit:** Ensures testing follows predefined methodologies (e.g., Agile, Waterfall).
    - **Compliance Audit:** Validates adherence to industry standards like HIPAA or GDPR.
    - **Security Audit:** Checks for vulnerabilities and security controls